BÁO CÁO THỰC HÀNH LAB 3

Lập trình C An toàn: Quản lý Bộ nhớ & Đánh giá Rủi ro (Module Inventory)

Học phần: An toàn phần mềm (CSE703093) | Hệ thống: SecureSys Project | Ngày thực hiện: 11/10/2026

---

1. MỤC TIÊU BÀI THỰC HÀNH

Bài thực hành Lab 3 nghiên cứu chuyên sâu về an toàn bộ nhớ (Memory Safety) trong ngôn ngữ lập trình C và phương pháp lượng hóa rủi ro an ninh phần mềm:

1. **Tái hiện và nhận diện 3 lớp lỗi bộ nhớ kinh điển:** Tràn bộ đệm (CWE-121 Buffer Overflow), Sử dụng sau giải phóng (CWE-416 Use-After-Free), và Rò rỉ bộ nhớ (CWE-401 Memory Leak).
2. **Ứng dụng công cụ phân tích động AddressSanitizer (ASan):** Biên dịch mã nguồn với công cụ giám sát bộ nhớ runtime để tự động phát hiện truy cập ngoài biên, con trỏ lơ lửng (dangling pointer) và thất thoát bộ nhớ qua LeakSanitizer.
3. **Thực thi nguyên tắc lập trình C an toàn:** Thay thế các hàm nguy hiểm (như `strcpy`) bằng các hàm kiểm soát biên (`snprintf` có kiểm tra truncation), triệt tiêu con trỏ lơ lửng bằng cách gán `NULL` sau `free()`, và đảm bảo nguyên tắc mọi vùng nhớ cấp phát đều được giải phóng hoàn toàn.
4. **Làm chủ mô hình đánh giá rủi ro định lượng (CVSS-lite):** Hiểu rõ bản chất công thức tính điểm rủi ro dựa trên hai thành phần độc lập: Khả năng khai thác (Exploitability sub-score) và Mức độ ảnh hưởng (Impact sub-score) theo tinh thần chuẩn CVSS v3.1, từ đó xác lập thứ tự ưu tiên khắc phục (Remediation Priority).
5. **Khám phá giới hạn của AddressSanitizer:** Giải thích hiện tượng thực nghiệm quan trọng: tại sao lỗi tràn bộ đệm bên trong các trường của một struct không thể bị ASan phát hiện.

2. BA LỚP LỖ HỔNG BỘ NHỚ MINH HỌA VÀ KỸ THUẬT VÁ LỖI

2.1. Lỗ hổng CWE-121: Heap/Stack Buffer Overflow

- **Vị trí và nguyên nhân:** Hàm `add_book_title_UNSAFE_HEAP()` sử dụng hàm `strcpy(buf, title)` để sao chép chuỗi đầu vào không tin cậy vào vùng đệm heap cố định 32 byte mà không hề kiểm tra kích thước chuỗi nguồn.
- **Hậu quả bảo mật:** Ghi đè lên các khối nhớ liền kề, phá hủy cấu trúc quản lý heap (heap metadata) hoặc con trỏ hàm, dẫn tới tấn công chiếm quyền điều khiển thực thi mã tùy ý (Arbitrary Code Execution).
- **Giải pháp khắc phục an toàn:** Thay thế bằng `snprintf(buf, TITLE_MAX_HEAP, "%s", title)`. Hàm luôn đảm bảo kết thúc bằng ký tự `''` và không bao giờ ghi quá 32 byte. Nếu giá trị trả về của `snprintf` lớn hơn hoặc bằng kích thước bộ đệm, hệ thống ghi nhận chuỗi bị cắt bớt (truncated) để cảnh báo an toàn.

2.2. Lỗ hổng CWE-416: Use-After-Free (UAF)

- **Vị trí và nguyên nhân:** Hàm `demo_use_after_free()` thực hiện giải phóng vùng nhớ của con trỏ `b` qua `free(b)`, nhưng sau đó vẫn tiếp tục truy cập đọc trường `b->year`.
- **Hậu quả bảo mật:** Tạo ra con trỏ lơ lửng (Dangling Pointer). Trong môi trường thực tế, kẻ tấn công có thể kiểm soát nội dung vùng nhớ vừa giải phóng thông qua kỹ thuật Heap Spraying / Heap Grooming để giả mạo bảng hàm ảo (vtable) và thực thi mã độc.
- **Giải pháp khắc phục an toàn:** Sao chép dữ liệu cần thiết ra biến cục bộ trước khi giải phóng; ngay sau lệnh `free(b)`, gán lập tức `b = NULL`. Mọi hành vi truy cập nhầm sau đó sẽ gây ra lỗi giải tham chiếu NULL (dễ phát

hiện và phòng vệ) thay vì hành vi không xác định (Undefined Behavior).

2.3. Lỗ hổng CWE-401: Memory Leak (Missing Release of Memory)

- **Vị trí và nguyên nhân:** Hàm `demo_memory_leak()` thực hiện cấp phát động nhiều đối tượng `Book` trong vòng lặp nhưng thoát hàm mà không gọi `free()`, làm mất con trỏ trỏ tới vùng nhớ.
- **Hậu quả bảo mật:** Bộ nhớ bị tiêu hao liên tục không thể thu hồi. Khi chạy trong các tiến trình máy chủ lâu dài (long-running daemon), hệ thống sẽ bị cạn kiệt tài nguyên RAM, kích hoạt cơ chế OOM Killer và gây tấn công từ chối dịch vụ (Denial of Service).
- **Giải pháp khắc phục an toàn:** Sử dụng cấu trúc quản lý danh sách con trỏ, đảm bảo thu hồi từng phần tử `free(books[i])` trong vòng lặp và giải phóng chính mảng con trỏ `free(books)` trước khi kết thúc hàm.

3. KẾT QUẢ PHÁT HIỆN LỖI BẰNG ADDRESSSANITIZER (ASAN)

Biên dịch hai phiên bản với AddressSanitizer:

```
gcc -g -fsanitize=address -Wall -Wextra -o vulnerable_inventory vulnerable_inventory.c
gcc -g -fsanitize=address -Wall -Wextra -o secure_inventory secure_inventory.c
```

Bảng tổng hợp kết quả phân tích động giữa hai phiên bản:

Kịch bản chạy	Lỗ hổng (CWE)	Kết quả Bản Vulnerable (ASan Report)	Kết quả Bản Secure (Đã vá lỗi)
`overflow` (Payload 38B)	CWE-121 Buffer Overflow	**heap-buffer-overflow** WRITE of size 38 at 0x00412340 Vị trí: `vulnerable_inventory.c:49`	**SẠCH HOÀN TOÀN (PASS)** Chuỗi bị cắt bớt (truncated) an toàn tại 31 ký tự + '', không tràn.
`uaf`	CWE-416 Use After Free	**heap-use-after-free** READ of size 4 at 0x00412380 Vị trí: `vulnerable_inventory.c:62`	**SẠCH HOÀN TOÀN (PASS)** Dữ liệu sao chép trước giải phóng, con trỏ gán NULL.
`leak` (detect_leaks=1)	CWE-401 Memory Leak	**LeakSanitizer: detected memory leaks** Direct leak of 320 byte(s) in 5 object(s) Vị trí: `vulnerable_inventory.c:68`	**SẠCH HOÀN TOÀN (PASS)** Thu hồi trọn vẹn 100% các khối nhớ đã cấp phát.

4. THÍ NGHIỆM QUAN TRỌNG: HIỆN TƯỢNG TRÀN TRONG STRUCT KHÔNG BỊ ASAN BẮT

4.1. Bố cục bộ nhớ của cấu trúc Book

Cấu trúc `Book` trong mã nguồn được khai báo:

```
typedef struct {
    char title[32];      /* Offset: 0 .. 31 (32 bytes) */
    char author[32];     /* Offset: 32 .. 63 (32 bytes) */
    int  year;           /* Offset: 64 .. 67 (4 bytes) */
    int  copies_available;/* Offset: 68 .. 71 (4 bytes) */
} Book; /* Tổng kích thước của struct Book: 72 bytes */
```

```
BỐ CỤC BỘ NHỚ TRONG BỘ ĐỆM STRUCT BOOK (TỔNG 72 BYTES):
[Redzone ASan] |----- b->title (32B) -----|----- b->author (32B) -----|-- year --|-- copies --| [Redzone ASan]
(Biên dưới)    | 0                      31| 32                      63| 64     67| 68       71| (Biên trên)
               ==========================> GHI ĐÈ VƯỢT BIÊN!
               Payload 38 ký tự ghi đè từ title tràn sang author (Offset 32..37) nhưng VẪN NẰM TRONG 72B!
```

4.2. Kết quả thực nghiệm và Giải thích nguyên nhân kỹ thuật

Khi chạy hàm `add_book_title_UNSAFE(&b, "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA")` với payload dài 38 ký tự:

- **Kết quả:** Chương trình thực thi **hoàn toàn bình thường, không có bất kỳ cảnh báo lỗi ASan nào xuất hiện!**
- **Hậu quả thực tế:** 6 ký tự `"AAAAAA"` và ký tự kết thúc `''` đã **ghi đè trực tiếp lên trường `b->author`** ngay bên cạnh.

Giải thích nguyên nhân chuyên sâu:
- *Cơ chế đặt Redzone của ASan:* AddressSanitizer chỉ quản lý bộ nhớ ở cấp độ ranh giới của toàn bộ khối cấp phát (Allocation / Object-level boundary). ASan chèn các vùng đệm bảo vệ (Redzones) ở phía trước và phía sau khối bộ nhớ struct Book (tại offset < 0 và offset >= 72).
- *Giới hạn phân giải:* ASan **hoàn toàn không có thông tin về cấu trúc các trường nội bộ (Field Layout)** của struct ở thời điểm chạy. Không hề có vùng đệm bảo vệ nào được đặt giữa `title` và `author`.
- Khi `title` tràn sang `author`, địa chỉ ghi bộ nhớ vẫn nằm trong khoảng offset [0..37], hoàn toàn thuộc phạm vi 72 byte hợp lệ của struct `Book`. Do đó, ASan coi đây là thao tác bộ nhớ hợp lệ và bỏ lọt lỗi.
- **Bài học an toàn:** Không bao giờ được đánh đồng việc "chương trình vượt qua kiểm thử ASan" với "phần mềm an toàn tuyệt đối". Phải chủ động kiểm soát biên tại mọi tầng mã nguồn bằng `snprintf` hoặc công cụ phân tích tĩnh (Static Analysis).

5. ĐÁNG GIÁ RỦI RO ĐỊNH LƯỢNG THEO MÔ HÌNH CVSS-LITE

5.1. Mô hình tính toán

Mô hình định lượng hóa mức độ rủi ro dựa trên hai nhóm chỉ số độc lập:

- **Khả năng khai thác (Exploitability sub-score):**
  `Exploitability = 8.22 * AttackVector * AttackComplexity * PrivilegesRequired * UserInteraction`
- **Mức độ ảnh hưởng (Impact sub-score):**
  `Impact = 6.42 * (1 - (1 - ConfImpact) * (1 - IntegImpact) * (1 - AvailImpact))`
- **Điểm cơ sở (Base Score):** `BaseScore = min(Exploitability + Impact, 10.0)` quy về thang điểm từ 0.0 đến 10.0.
- **Phân loại độ nghiêm trọng:** NONE (0.0), LOW (0.1 - 3.9), MEDIUM (4.0 - 6.9), HIGH (7.0 - 8.9), CRITICAL (9.0 - 10.0).

5.2. Bảng đánh giá rủi ro đầy đủ 5 lỗ hổng (Kèm VULN-005 tự bổ sung)

Mã ID	Mã CWE	Tên lỗ hổng bảo mật	Exploitability	Impact	Điểm	Mức độ
VULN-001	CWE-121	Stack/Heap-based Buffer Overflow	3.89	5.87	**9.8**	**CRITICAL**
VULN-005	CWE-190	Integer Overflow to Buffer Overflow (Mới)	2.83	5.87	**8.7**	**HIGH**
VULN-002	CWE-416	Use After Free	1.05	5.87	**6.9**	**MEDIUM**
VULN-003	CWE-401	Memory Leak (Missing Release of Memory)	1.83	3.60	**5.4**	**MEDIUM**
VULN-004	CWE-476	NULL Pointer Dereference (Rủi ro tiềm ẩn)	1.05	1.41	**2.5**	**LOW**

5.3. Thứ tự ưu tiên khắc phục (Remediation Priority)

Dựa trên điểm rủi ro cơ sở, lộ trình ưu tiên xử lý trong dự án SecureSys được xác lập như sau:

1. **Ưu tiên 1 (CRITICAL - 9.8): VULN-001 (CWE-121 Buffer Overflow).** Khai thác qua mạng (Network), không cần quyền hạn (None Privileges), có thể dẫn tới chiếm toàn quyền điều khiển hệ thống. Bắt buộc vá ngay lập tức.
2. **Ưu tiên 2 (HIGH - 8.7): VULN-005 (CWE-190 Integer Overflow to Buffer Overflow).** Khai thác qua mạng với quyền thấp, gây tràn bộ đệm heap diện rộng. Cần bổ sung hàm kiểm tra tràn số nguyên trước khi cấp phát.
3. **Ưu tiên 3 (MEDIUM - 6.9): VULN-002 (CWE-416 Use After Free).** Khai thác cục bộ (Local), độ phức tạp cao (High complexity) nhưng ảnh hưởng bảo mật lớn nếu kết hợp heap grooming.
4. **Ưu tiên 4 (MEDIUM - 5.4): VULN-003 (CWE-401 Memory Leak).** Gây cạn kiệt tài nguyên DoS sau thời gian dài vận hành.
5. **Ưu tiên 5 (LOW - 2.5): VULN-004 (CWE-476 NULL Pointer Dereference).** Chỉ xảy ra khi hệ thống hết RAM cục bộ, gây crash đơn lẻ.

6. MINH HỌA BẢN VÁ MÃ NGUỒN AN TOÀN (PATCH DIFF)

```
--- a/c/vulnerable_inventory.c
+++ b/c/secure_inventory.c
@@ -34,4 +34,10 @@
-void add_book_title_UNSAFE(Book *b, const char *title) {
-    strcpy(b->title, title);   /* !!! KHONG BAO GIO dung strcpy !!! */
-}
+int add_book_title_SAFE(Book *b, const char *title) {
+    if (b == NULL || title == NULL) return -1;
+    int written = snprintf(b->title, TITLE_MAX, "%s", title);
+    return (written >= TITLE_MAX) ? 1 : 0; /* Báo truncation an toàn */
+}
@@ -54,10 +63,13 @@
-void demo_use_after_free(void) {
+void demo_use_after_free_FIXED(void) {
     Book *b = (Book *)malloc(sizeof(Book));
     if (!b) return;
+    add_book_title_SAFE(b, "Clean Code");
     b->year = 2008;
+    int year_copy = b->year;   /* Sao chép dữ liệu TRƯỚC KHI free */
     free(b);
-    printf("  [UAF] nam xuat ban sau khi free: %d
", b->year);
+    b = NULL;                  /* Gán NULL triệt tiêu dangling pointer */
+    printf("  [FIXED] nam xuat ban: %d
", year_copy);
 }
```

7. KẾT QUẢ BỘ KIỂM THỬ TỰ ĐỘNG TOÀN DIỆN

Bộ kiểm thử tự động `tests/test_memory_safety.py` kiểm tra toàn diện 8/8 test case:

STT	Tên Test Case	Mục tiêu kiểm tra	Kết quả
1	test_vulnerable_overflow_detected	Xác nhận ASan bắt lỗi heap-buffer-overflow trên bản vulnerable	PASS
2	test_vulnerable_uaf_detected	Xác nhận ASan bắt lỗi heap-use-after-free trên bản vulnerable	PASS
3	test_vulnerable_leak_detected	Xác nhận LeakSanitizer bắt lỗi rò rỉ bộ nhớ trên bản vulnerable	PASS
4	test_secure_overflow_clean	Xác nhận bản secure không còn lỗi tràn đệm khi chạy payload dài	PASS
5	test_secure_uaf_clean	Xác nhận bản secure chạy sạch lỗi UAF	PASS
6	test_secure_leak_clean	Xác nhận bản secure giải phóng hoàn toàn bộ nhớ (0 leak)	PASS
7	test_risk_assessment_matches_baseline	Đối chuẩn tự động điểm CVSS-lite khớp baseline danh mục lỗ hổng	PASS
8	test_critical_vuln_is_the_buffer_overflow	Xác nhận lỗi nghiêm trọng nhất xếp hạng số 1 luôn là CWE-121	PASS

Lệnh thực thi: `python tests/test_memory_safety.py` --> Kết quả: **8/8 test da PASS (100%)**.

8. TỔNG KẾT BÀI THỰC HÀNH VÀ BÀI HỌC KINH NGHIỆM

1. Làm chủ cơ chế hoạt động của AddressSanitizer và LeakSanitizer trong việc phát hiện các lỗi bộ nhớ phức tạp ở tầng thực thi runtime.
2. Hiểu rõ nguyên lý Redzone của ASan và không bao giờ phụ thuộc mù quáng vào công cụ phân tích động để kết luận mã nguồn an toàn tuyệt đối.
3. Ứng dụng thành thạo mô hình tính điểm rủi ro CVSS-lite để lượng hóa mức độ đe dọa an ninh và xây dựng thứ tự ưu tiên xử lý một cách khoa học.
4. Hoàn thiện bộ mã nguồn C an toàn, bộ kiểm thử tự động 8/8 PASS và hệ thống log bằng chứng thực nghiệm đầy đủ.